

Wzorzec projektowy: Iterator

Ujednolicone przechodzenie po kolekcjach danych

Karolina Nycz

Dlaczego przeglądanie danych może być problemem?

W aplikacjach często korzystamy z różnych kolekcji danych – list, tablic czy innych struktur.

Każda z nich może mieć inny sposób dostępu do elementów.

Powoduje to:

- konieczność pisania różnych pętli
- większe ryzyko błędów (np. złe indeksy)
- trudniejszy do utrzymania kod

W większych projektach szybko robi się z tego bałagan.

Rozwiązanie: Iterator

Zamiast operować bezpośrednio na strukturze danych, **używamy iteratora**, który:

- udostępnia jednolity sposób przechodzenia po elementach
- oddziela logikę przechodzenia od samej kolekcji
- upraszcza korzystanie z danych

Dzięki temu sposób przeglądania elementów jest zawsze taki sam.

Przykład działania

Wyobraźmy sobie aplikację zarządzającą listą produktów:

- dane mogą być przechowywane w różnych kolekcjach
- każda struktura może działać inaczej
- iterator pozwala przechodzić po wszystkich elementach w ten sam sposób

Np. używając pętli **foreach**, niezależnie od typu kolekcji.

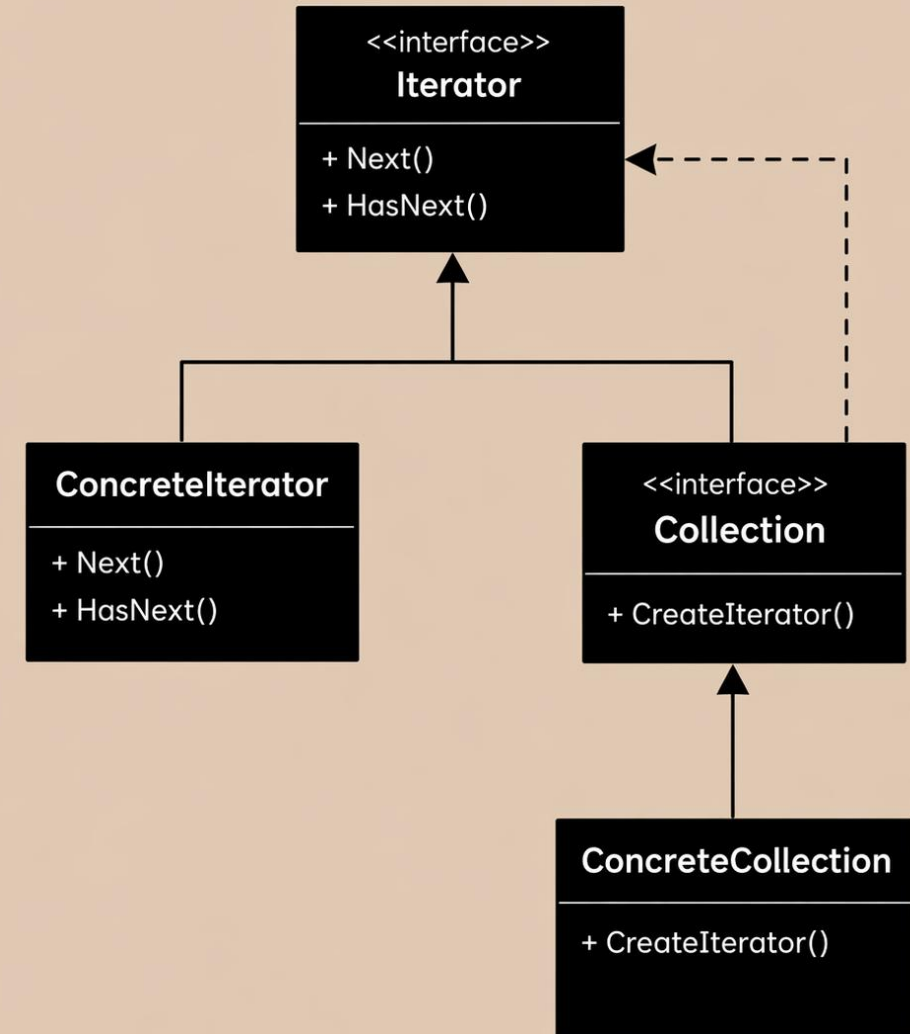
Struktura wzorca

Wzorzec Iterator składa się z następujących elementów:

- **Iterator** – definiuje metody przechodzenia (np. pobranie kolejnego elementu)
- **ConcreteIterator** – implementuje konkretny sposób iteracji
- **Kolekcja** – przechowuje dane
- **Metoda tworząca iterator** – umożliwia uzyskanie iteratora

Dzięki temu klient nie musi znać szczegółów działania kolekcji.

Schemat UML



Przykładowy fragment kodu C#

```
using System;
using System.Collections.Generic;

class BookCollection : IEnumerable<string>
{
    private List<string> books = new List<string>();
    public void Add(string book) => books.Add(book);
    public IEnumerator<string> GetEnumerator() => books.GetEnumerator();
}

class Program
{
    static void Main()
    {
        var library = new BookCollection();
        library.Add("1984");
        library.Add("Proces");
        library.Add("Dżuma");

        foreach (var book in library)
            Console.WriteLine(book);
    }
}
```

Zalety i wady

Zalety:

- jeden sposób przeglądania różnych kolekcji
- lepsza organizacja kodu
- oddzielenie logiki od danych
- mniejsze ryzyko błędów

Wady:

- dodatkowa warstwa abstrakcji
- nie zawsze potrzebny przy bardzo prostych przypadkach

Kiedy stosować Iterator?

Wzorzec Iterator sprawdza się, gdy:

- pracujemy na różnych strukturach danych
- chcemy uprościć dostęp do elementów
- zależy nam na czytelności kodu
- chcemy oddzielić sposób przechodzenia od danych
- Szczególnie przydatny w aplikacjach wykorzystujących kolekcje i przetwarzanie danych